Amjad S Althabteh

Amjadjamalshadi@gmail.com | 734-657-3714 | Linkedin | Portfolio | Github | X | Grand Forge Games

SUMMARY

Performance-focused C++ systems engineer specializing in low-latency architecture, simulation systems, and graphics/rendering pipelines. Focused on GPU throughput, memory layout, cache efficiency, and profiling-driven optimization, from lock-free concurrent systems to custom allocators and rendering engines.

EDUCATION

Wayne State University — ***Detroit, MI***
B.S. in Computer Science — **GPA: 3.76**

EXPERIENCE

Immersalab Graphics | *Detroit, MI* — ***05/25 – 02/26***
Graphics Engineer
- Reduced GPU idle time by ~30% using RenderDoc profiling and rendering pipeline optimizations.
- Designed internal C++/JSON tooling and asset pipeline; profiled and optimized GLSL shader compilation paths.
- Improved frame submission throughput and GPU resource utilization across rendering workloads.

UE Game Programmer C++| Yorktown, VA — **04/26 – 08/26**
Grand Forge Games
- Built a gameplay ability system in C++ using Unreal Engine GAS, supporting ability types with Blueprint integration.
- Reduced animation transition overhead using multi-layer blending and reusable C++ systems.

Rare Munchies | *Ann Arbor, MI* — ***09/23 – 04/24***
Software Development Intern
- Reduced unnecessary re-renders by ~35% through improved state management and component optimization.
- Built reusable TypeScript component systems focused on runtime efficiency and minimal overhead.

Wayne State University | *Detroit, MI* — 0*2/25 – 12/25*
Undergraduate Researcher | Orbital Collision Modeling
- Optimized collision-detection performance by ~30% through spatial partitioning, allowing simulations to scale efficiently across large orbital datasets.
- Designed a spatial-partitioning scheme that reduced simulation runtime by ~30%, improving scalability for large-scale N-body orbital collision analysis.

PROJECTS

Glint 3D Rendering Engine | *C++17/OpenGL* | glint rendering engine v 01 — ***Glint-Rendering***
- Designed a memory pool allocator using placement new and free list reuse, reducing heap allocation overhead by ~60%.
- Improved rendering performance using frustum culling and GPU instanced rendering, cutting draw calls by ~40%.
- Used L1/L2 cache warming to preload key asset data before rendering and reduce cold-start latency.

IronPath Robotic Control Systems | *IronPath Control Systems* | *Rust* | — ***Robotic-ironpath***
- Achieved a 100Hz control loop with ~142µs average latency (p99: 540µs), tracked via custom CSV telemetry and latency..
- Implemented custom allocation tracking using overloaded new/delete operators for memory leak detection.
-

3D Space Simulation Engine | *C++* | *prediction-model-orbital-collision c++* — ***OrbitalCollision***
- Engineered a modular C++17/OpenGL simulation engine simulating 50+ orbital bodies across 1M+ trajectories.
- Reduced simulation runtime by ~40% using spatial partitioning and numerical integration.
- Used loop unrolling and SIMD-friendly vector operations to reduce branching in tight performance loops.

HFT OrderBook | *C++20* | *Low-Latency Performance System* — ***HFT-lowlatency***
- Built a C++20 order matching engine with price-time priority achieving microsecond-level order resolution.
- Eliminated false sharing using cache-aligned (alignas(64)) structures, reducing memory contention under concurrent load.

SKILLS

Languages: (C++11–C++23), Rust, C, C#, Python, TypeScript, Java, Javascript, SQL, Assembly

Graphics & Engines: OpenGL, GLSL, Unreal Engine, Unity, Godot, vulkan

Systems: Performance**,** custom allocators, memory-pools, multithreading, lock-free ds, cache optimization, profiling, linux